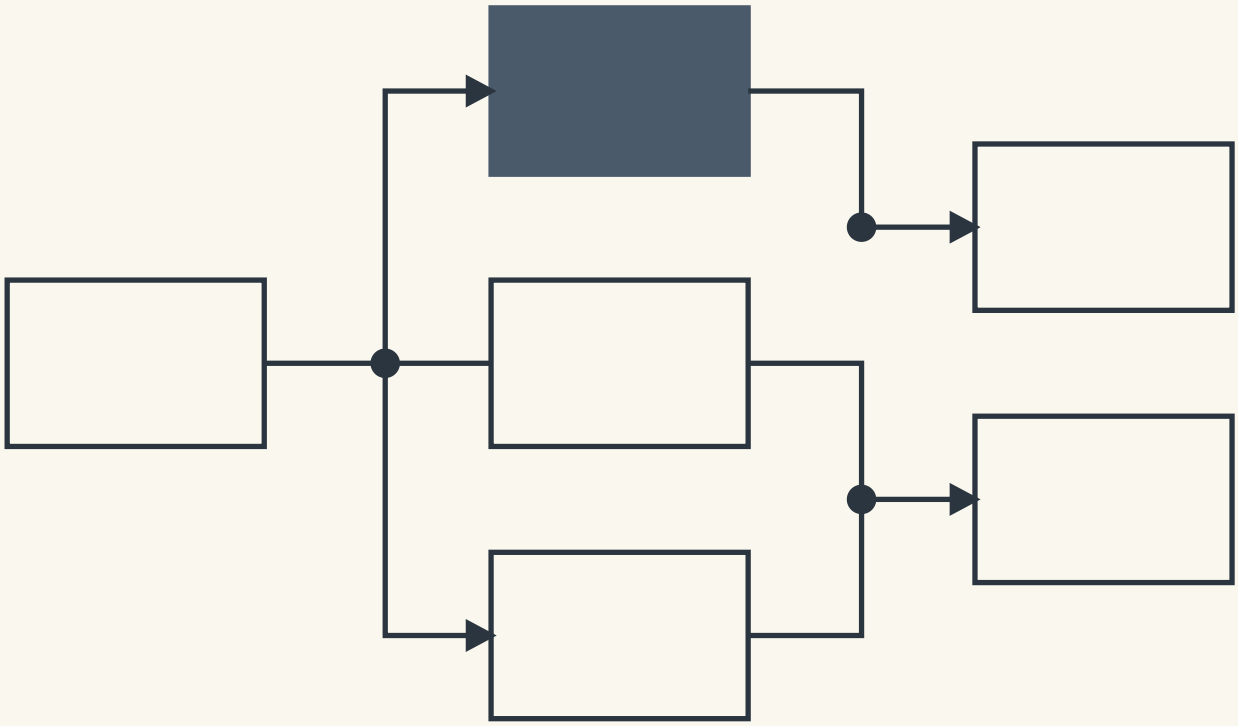
ENSAIOS TÉCNICOS · Nº 7



Arquitetura de software

notas de campo

Decisões, trade-offs e o que se aprende entregando sistemas em produção

Rafael Antunes Lobo

Por que estas notas

Estas notas reúnem decisões de arquitetura de software tomadas em projetos reais, com o raciocínio por trás de cada escolha e o que deu certo ou errado depois que o sistema entrou em produção. Não é um catálogo de padrões: é um caderno de campo, escrito para quem precisa decidir com informação incompleta, prazo curto e uma equipe de verdade.

Arquitetura, aqui, é o conjunto de decisões que custam caro para mudar depois. Escolher um framework de interface quase nunca é uma delas; escolher onde ficam os dados e como os módulos conversam, quase sempre é.

1. Comece pelo problema, não pelo diagrama

O erro mais comum que vi em revisões de projeto foi o diagrama bonito que resolve o problema errado. Antes de desenhar caixas, escreva em uma página: quem usa o sistema, quais são as três operações mais frequentes, qual o volume esperado no primeiro ano e o que acontece com o negócio se o sistema ficar fora do ar por uma hora.

Essas quatro respostas eliminam metade das alternativas. Um sistema interno usado por quarenta pessoas em horário comercial não precisa da mesma estratégia de disponibilidade de uma loja virtual na véspera de feriado.

Requisitos de qualidade, por escrito

Desempenho, segurança, custo e facilidade de manutenção puxam em direções diferentes. Peça ao time de negócio para ordenar esses atributos. A ordem vai mudar no meio do projeto, e tudo bem: o importante é que a mudança seja explícita.

2. Monólito modular antes de microsserviços

Microsserviços resolvem um problema organizacional, o de muitas equipes mexendo no mesmo código, e cobram por isso um preço operacional alto: rede entre chamadas, implantação independente, rastreamento distribuído e consistência eventual. Equipes pequenas quase sempre ganham mais com um monólito bem dividido em módulos.

- Cada módulo tem sua pasta, seus testes e uma interface pública pequena.
- Módulos não leem as tabelas uns dos outros; pedem os dados pela interface.
- Dependências entre módulos são verificadas automaticamente no build.

Quando um módulo realmente precisar escalar sozinho ou ser entregue por outra equipe, ele já terá fronteiras claras, e extraí-lo vira um trabalho de semanas, não de meses.

3. Fronteiras e contratos

Toda conversa entre partes do sistema é um contrato, mesmo quando ninguém escreveu. Prefira contratos explícitos: esquemas de API versionados, eventos com formato documentado e mensagens de erro que digam o que fazer.

Síncrono ou assíncrono

Chamadas síncronas são simples de entender e de depurar, mas acoplam a disponibilidade de quem chama à de quem é chamado. Filas e eventos desacoplam no tempo, ao custo de mais peças para monitorar. Uma regra prática: use chamadas síncronas quando o usuário está esperando a resposta e eventos quando o trabalho pode acontecer depois.

> *Toda fila precisa de alguém olhando para ela. Uma fila que cresce em silêncio é um incidente com data marcada.*

4. Dados: a parte mais difícil de mudar

Código se reescreve; dados se migram, e migração de dados em produção é onde os projetos sangram. Gaste tempo no modelo: nomes claros, chaves estrangeiras de verdade, restrições no banco e não só na aplicação.

1. Defina quem é o dono de cada informação. Duas fontes da verdade viram nenhuma.
2. Guarde datas com fuso horário e valores monetários sem ponto flutuante.
3. Planeje como apagar dados desde o início: retenção também é requisito.
4. Toda migração precisa de um caminho de volta testado.

5. Observabilidade desde o primeiro dia

Um sistema que você não consegue observar é um sistema que você não consegue operar. Desde a primeira entrega, tenha logs estruturados com um identificador por requisição, métricas das operações principais e alertas que acordem alguém só quando o usuário estiver sendo afetado.

Painéis bonitos ajudam pouco se ninguém sabe o que é normal. Anote os valores de referência de cada métrica logo nas primeiras semanas.

6. Registre as decisões

Um registro de decisão de arquitetura (ADR) é um documento curto que responde: o que decidimos, em que contexto, que alternativas consideramos e quais consequências aceitamos. Ele evita que a mesma discussão aconteça a cada novo integrante da equipe.

Campo	O que escrever
Título	A decisão em uma frase, no imperativo
Contexto	As forças em jogo: prazos, volume, equipe, restrições
Alternativas	Duas ou três opções reais, com prós e contras
Decisão	A escolha e o motivo principal
Consequências	O que fica mais fácil, o que fica mais difícil e quando revisar

Conclusão

Boa arquitetura raramente é a mais sofisticada. É a que permite à equipe entregar com segurança hoje e mudar de ideia amanhã sem reescrever tudo. Decida tarde o que pode esperar, decida cedo o que custa caro mudar e escreva as duas coisas.